

## 一、背景介绍

本次作业的目的是利用一系列与房价相关的因素，例如房屋地址、类型等，训练出一个房价预测模型。从问题的描述可以看出这是一个经典的机器学习模型。为了解决这个问题，我们需要对历史数据进行分析，构建预测模型，从而预测未来的房价。

## 二、数据探索性分析

从train.csv中我们可以看出，LAND SQUARE、GROSS SQUARE、YEAR BUILT 三列是存在数据缺失的情况的。同时，有些数据并非可以直接送入模型学习的数字信息，而是文本信息，需要利用one-hot编码对他们进行转化。

## 三、特征工程

我使用了随机森林和lightGBM两种模型的混合模型进行预测。由于这两个模型的特征工程并不相同，所以我会将它们都展示出来。我将原本train.csv中的数据集记为train\_and\_valid\_X，表示其既包含训练数据集也包含测试数据集。然后我们将train\_and\_valid\_X与test\_X合并，记为data\_X\_rf和data\_X\_lgb，分别用于两个模型的特征工程。

```
data_X_rf = pd.concat([train_and_valid_X, test_X]) # 用于训练随机森林
data_X_lgb = data_X_rf.copy() # 用于训练梯度提升
```

### 3.1 删除无效特征

对两个数据集，我们分别删去如下特征(特征的重要性主要出于我的主观判断，因此很可能不准确)。

```
del data_X_lgb['ADDRESS']
del data_X_lgb['APARTMENT NUMBER']
del data_X_lgb['BUILDING CLASS AT PRESENT']
del data_X_lgb['SALE DATE']
del data_X_lgb['LOT']
del data_X_lgb['RESIDENTIAL UNITS']
del data_X_lgb['COMMERCIAL UNITS']

del data_X_rf['ADDRESS']
del data_X_rf['APARTMENT NUMBER']
del data_X_rf['BUILDING CLASS AT PRESENT']
del data_X_rf['BUILDING CLASS AT TIME OF SALE']
del data_X_rf['NEIGHBORHOOD']
del data_X_rf['SALE DATE']
```

```
del data_X_rf['LAND SQUARE FEET']
del data_X_rf['GROSS SQUARE FEET']
```

### 3.2 处理缺失数据

由于data\_X\_rf不再有缺失数据，因此我们只用对data\_X\_lgb进行缺失数据处理。对于LAND SQUARE、GROSS SQUARE 和 YEAR BUILT 三个数据字段，我们将 ‘-’ 和 ‘0’ 替换为np.nan，并用该列的中位数取代无效值。在这之后，我们对这三列进行标准化处理。

```
# 处理缺失数据
missing_data_column = ['LAND SQUARE FEET', 'GROSS SQUARE FEET', 'YEAR BUILT']
for column in missing_data_column:
    # 用同行政区(BOROUGH)同建筑类别的中位数填充
    # 替换‘-’为 np.nan
    data_X_lgb[column] = data_X_lgb[column].replace(' - ', np.nan)
    data_X_lgb[column] = data_X_lgb[column].replace('0', np.nan)

    # 转成 float 类型
    data_X_lgb[column] = data_X_lgb[column].astype('float64')

    # 用中位数填充缺失值
    data_X_lgb[column] = data_X_lgb[column].fillna(data_X_lgb[column].median())
scaler = StandardScaler()

features_to_scale = missing_data_column
data_X_lgb[features_to_scale] =
scaler.fit_transform(data_X_lgb[features_to_scale].astype('float16'))
```

### 3.3 类别变量处理

正如之前提到的，许多文本数据并不利于直接送入模型，我们将之转化为分类变量，如下：

```
# Let's convert some of the columns to appropriate datatype
category_columns_1 = [
    'BOROUGH',
    'NEIGHBORHOOD',
    'BUILDING CLASS CATEGORY',
    'TAX CLASS AT PRESENT',
    'BLOCK',
    'ZIP CODE',
    'TAX CLASS AT TIME OF SALE',
    'BUILDING CLASS AT TIME OF SALE',
    'TOTAL UNITS',
]
category_columns_2 = [
    'BOROUGH',
```

```

    'BUILDING CLASS CATEGORY',
    'TAX CLASS AT PRESENT',
    'TAX CLASS AT TIME OF SALE'
]
for column in category_columns_1:
    data_X_lgb[column] = data_X_lgb[column].astype('category')
for column in category_columns_2:
    data_X_rf[column] = data_X_rf[column].astype('category')

```

随后，我们对这些分类变量进行one-hot编码，并用其代替原来的列

```

one_hot_features_1 = category_columns_1
one_hot_features_2 = category_columns_2

# Convert categorical variables into dummy/indicator variables (i.e. one-hot
encoding).
one_hot_encoded_1 = pd.get_dummies(data_X_lgb[one_hot_features_1])
one_hot_encoded_1.info(verbose=True, memory_usage=True)

one_hot_encoded_2 = pd.get_dummies(data_X_rf[one_hot_features_2])
one_hot_encoded_2.info(verbose=True, memory_usage=True)

# Replace the original column
data_X_lgb = data_X_lgb.drop(one_hot_features_1,axis=1)
data_X_lgb = pd.concat([data_X_lgb, one_hot_encoded_1] ,axis=1)

data_X_rf = data_X_rf.drop(one_hot_features_2,axis=1)
data_X_rf = pd.concat([data_X_rf, one_hot_encoded_2] ,axis=1)

```

### 3.4 重新提取数据集

我们将原本训练数据集的前6/7用于训练，后1/7用于混合模型的超参数确定，如下：

```

train_and_valid_X_lgb = data_X_lgb[:num_train_samples].to_numpy()
test_X_lgb = data_X_lgb[num_train_samples:].to_numpy()

cut = num_train_samples * 6 // 7
train_X_lgb = train_and_valid_X_lgb[:cut]
valid_X_lgb = train_and_valid_X_lgb[cut:]
train_y = train_and_valid_y[:cut]
valid_y = train_and_valid_y[cut:]

lgb_train = lgb.Dataset(train_X_lgb, train_y)
lgb_eval = lgb.Dataset(test_X_lgb, test_y, reference=lgb_train)

train_and_valid_X_rf = data_X_rf[:num_train_samples].to_numpy()
test_X_rf = data_X_rf[num_train_samples:].to_numpy()

train_X_rf = train_and_valid_X_rf[:cut]
valid_X_rf = train_and_valid_X_rf[cut:]

```

## 四、预测模型的建立

我们使用LightGBM和RandomForestRegressor的混合模型进行预测。先对两个模型分别进行训练

```
rf_regr = RandomForestRegressor()
rf_regr.fit(train_X_rf, train_y)
valid_pred_rf = rf_regr.predict(valid_X_rf)
```

```
params = {
    'num_leaves': 200,
    'feature_fraction': 0.25,
    'learning_rate': 0.03,
    'objective': 'mape'
}

lgb_t = lgb.train(params=params, train_set=lgb_train, num_boost_round=2000)
valid_pred_lgb = lgb_t.predict(valid_X_lgb)
```

随后利用验证集选取加权平均的超参数 $w$

```
result = []
for w in np.linspace(0, 1, 101):
    result.append((w, mean_absolute_percentage_error(valid_y, valid_pred_lgb * w +
    valid_pred_rf * (1 - w))))
w = (min(result, key=lambda x: x[1]))[0]
```

最后得到结果

```
Y_pred = lgb_t.predict(test_X_lgb) * w + rf_regr.predict(test_X_rf) * (1 - w)
print(mean_absolute_percentage_error(test_y, Y_pred))
```

## 五、结果的分析与总结

### 5.1 结果分析

在本次实验中，我们使用LightGBM和RandomForestRegressor的混合模型进行预测，最终结果如下：

特征工程有效性：通过处理无效数据、删除无用特征、处理地址列和日期列、标准化数值特征以及使用独热编码处理分类变量，我们成功地为模型提供了干净且有用的数据。这些步骤在提高模型准确性方面起到了关键作用。

模型选择：LightGBM 是一种高效的梯度提升框架，结合随机回归森林，组成混合模型，能得到比两者单独使用更好的效果。

参数调优：我们选择了合适的参数，如 `num_leaves`、`feature_fraction` 和 `learning_rate`。这些参数在一定程度上优化了模型的性能。

## 5.2 总结

上述实验仍有改进空间：

1. 特征选择：可以进一步探索和选择更有代表性的特征，或者通过特征工程生成新的特征。
2. 模型调优：可以尝试更多的模型参数组合，进一步优化模型性能。

本次实验展示了数据预处理和特征工程在构建高效预测模型中的重要性。通过合理的特征处理和模型选择，我们能够构建出较为准确的房价预测模型。